

Address Design & Pharmacy Recommendation Logic

Overview

This document explains the design decisions behind the **Address** entity and the algorithm used to recommend the best pharmacy to patients based on distance calculations.

Part 1: Address Entity Design

Why Address is a Separate Entity

Rather than storing address as a simple string attribute in the User class, Address is modeled as its own entity because:

- 1. **Rich Data Structure:** Address contains multiple related fields (geographic, administrative, and mapping data)
- 2. **Reusability:** Multiple entity types need addresses (Doctor clinic location, Patient home, Pharmacy location)
- 3. **Behavior & Methods:** Address performs operations like distance calculations and Google Maps validation
- 4. **Evolution:** Enables future features like address history, multiple addresses per user, or geocoding services
- 5. **Data Integrity:** Separation of concerns keeps location data normalized and consistent

Relationship to User

User "1" -- "0..*" Address

- **1:** Each User has one primary address (or none)
- **0..*:** A User can have multiple addresses (home, work, clinic location, etc.)

Part 2: Address Attributes & Purpose

Complete Attribute Breakdown

Attribute	Type	Purpose	Usage
AddressId	int	Primary Key	Database entity identification
Wilaya	string	Province/Region (Algerian administrative division)	Regional filtering, administrative queries
Province	string	Sub-regional area	More granular location hierarchy
Commune	string	Municipality/City	Local administrative unit

Attribute	Type	Purpose	Usage
Rue	string	Street name	Human-readable address component
NumMaison	int	Building/House number	Complete postal address formation
PlaceId	string	Google Maps unique identifier	Stable reference; survives address text changes
FormattedAddress	string	Full standardized address string	Display to users; geocoding validation
Lat	decimal	Latitude coordinate	Geographic point for distance calculations
Lng	decimal	Longitude coordinate	Geographic point for distance calculations

Attribute Categories

1. Administrative Fields (Local/Algerian Context)

Wilaya → Province → Commune → Rue + NumMaison

- Used for regional filtering and local business logic
- Example query: "Find all pharmacies in Wilaya = 'Algiers'"
- Supports hierarchical location queries

2. Geospatial Fields (Mathematical)

Lat, Lng

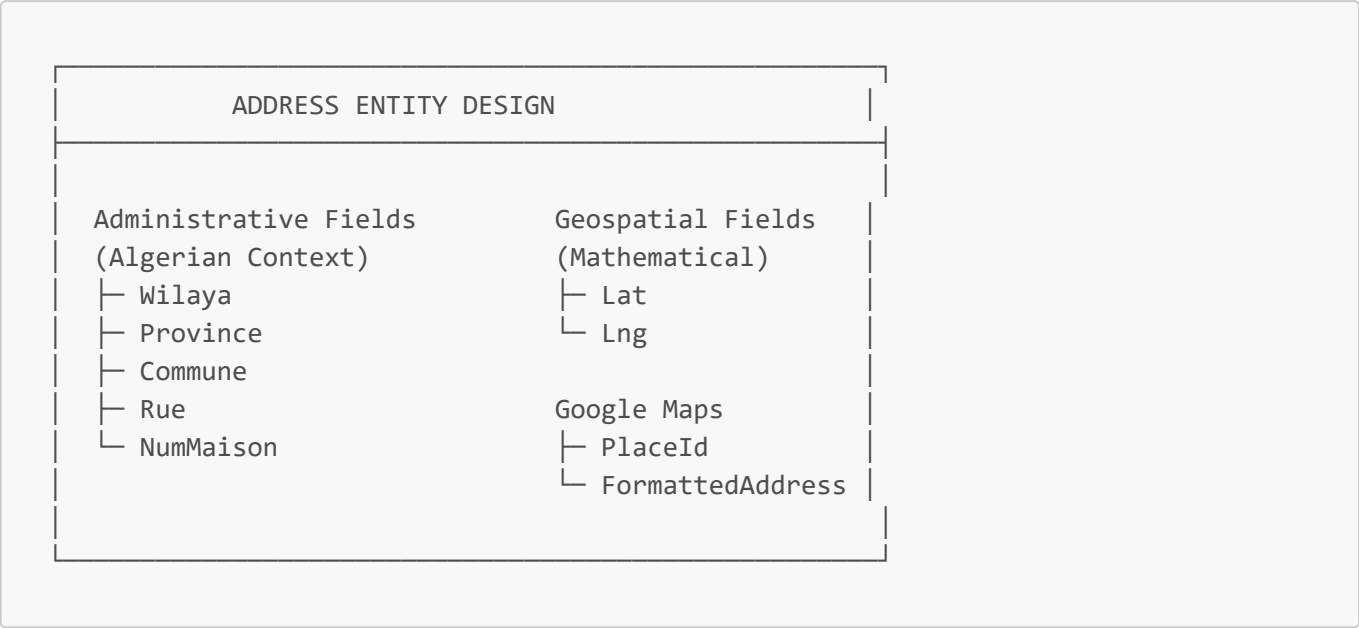
- Used for distance calculations (Haversine formula)
- Enable "nearest pharmacy" queries
- Support map visualizations
- Critical for recommendation algorithm

3. Google Maps Integration

PlaceId, FormattedAddress

- **PlaceId**: Immutable identifier from Google; survives address corrections
- **FormattedAddress**: Standardized format; validated by Google; displayed to users

Design Philosophy: Hybrid Approach



Why combine all three?

- **Administrative:** Faster regional queries without calculating distances
- **Geospatial:** Accurate distance-based recommendations
- **Google Maps:** Single source of truth for coordinate accuracy

Part 3: Distance Calculation Algorithm

Mathematical Foundation

The system uses the **Haversine Formula** to calculate the great-circle distance between two geographic points (latitude/longitude coordinates).

Haversine Formula

```
Given two points:
Point 1: (lat1, lng1)
Point 2: (lat2, lng2)

Step 1: Convert degrees to radians
lat1_rad = lat1 × π/180
lng1_rad = lng1 × π/180
lat2_rad = lat2 × π/180
lng2_rad = lng2 × π/180

Step 2: Calculate differences
Δlat = lat2_rad - lat1_rad
Δlng = lng2_rad - lng1_rad

Step 3: Apply Haversine formula
a = sin²(Δlat/2) + cos(lat1_rad) × cos(lat2_rad) × sin²(Δlng/2)
c = 2 × atan2(√a, √(1-a))
```

```
distance = R × c
```

Where:

R = Earth radius (6371 km)

distance = distance in kilometers

Simplified Approximation

For shorter distances (< 50 km), a simpler formula works:

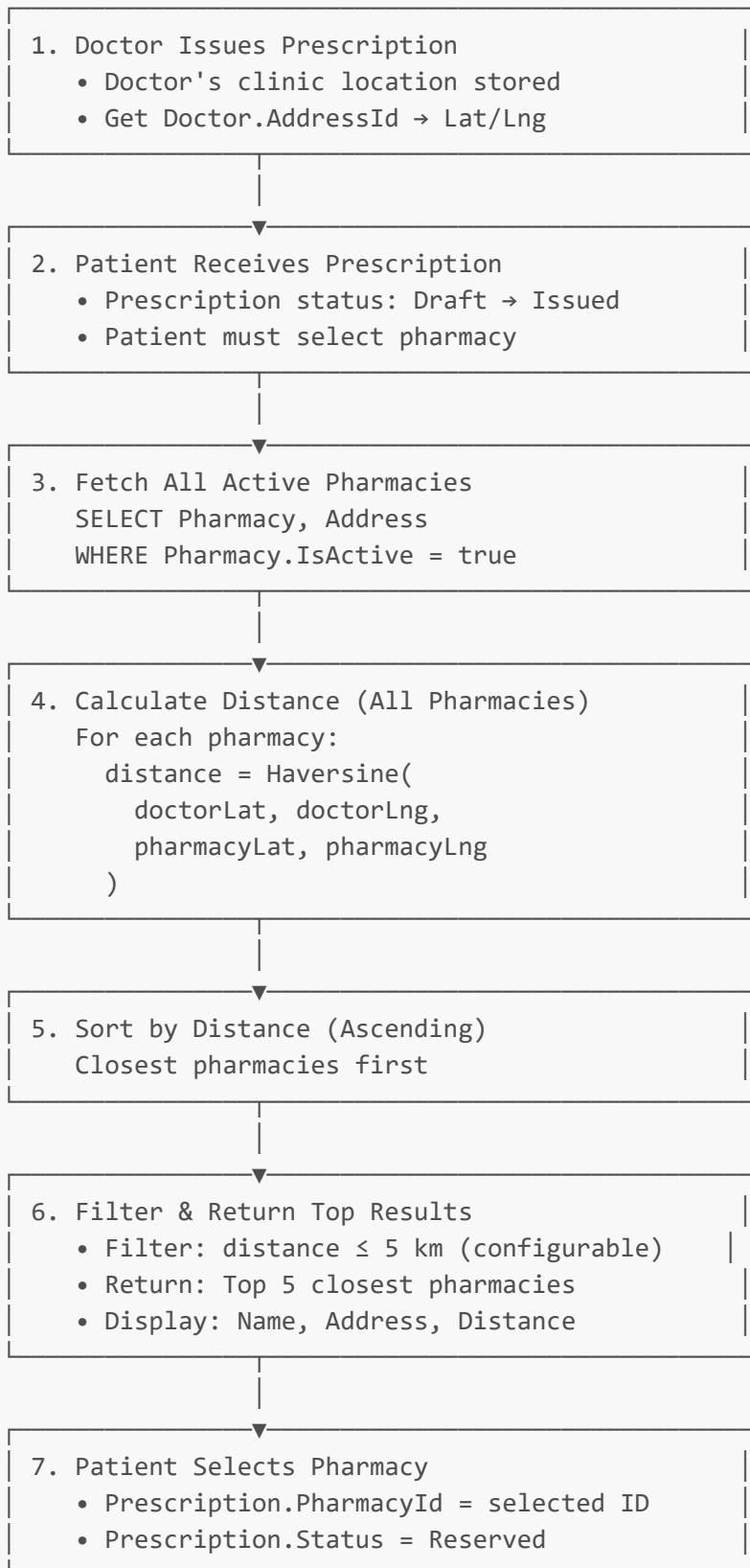
```
distance ≈ √[(lat2-lat1)² + (lng2-lng1)²] × 111 km  
(111 km ≈ 1 degree latitude)
```

Code Example (C#/.NET)

```
public static decimal CalculateDistance(decimal lat1, decimal lng1, decimal lat2,  
decimal lng2)  
{  
    const decimal R = 6371m; // Earth's radius in km  
  
    // Convert degrees to radians  
    var lat1Rad = lat1 * (decimal)Math.PI / 180m;  
    var lat2Rad = lat2 * (decimal)Math.PI / 180m;  
    var deltaLat = (lat2 - lat1) * (decimal)Math.PI / 180m;  
    var deltaLng = (lng2 - lng1) * (decimal)Math.PI / 180m;  
  
    // Haversine formula  
    var a = (decimal)Math.Sin((double)deltaLat / 2) *  
(decimal)Math.Sin((double)deltaLat / 2) +  
        (decimal)Math.Cos((double)lat1Rad) *  
(decimal)Math.Cos((double)lat2Rad) *  
        (decimal)Math.Sin((double)deltaLng / 2) *  
(decimal)Math.Sin((double)deltaLng / 2);  
  
    var c = 2m * (decimal)Math.Atan2((decimal)Math.Sqrt((double)a),  
(decimal)Math.Sqrt((double)(1 - a)));  
  
    return R * c;  
}
```

Part 4: Pharmacy Recommendation Algorithm

Complete Flow Diagram



SQL Query

```
-- Get recommended pharmacies for a prescription
SELECT
  TOP 5
```

```
p.PharmacyId,
p.Name,
a.FormattedAddress,
a.Commune,
a.Wilaya,
dbo.CalculateDistance(
    @doctorLat, @doctorLng,
    a.Lat, a.Lng
) AS DistanceKm
FROM Pharmacy p
INNER JOIN Address a ON p.AddressId = a.AddressId
WHERE
    p.IsActive = 1
    AND dbo.CalculateDistance(
        @doctorLat, @doctorLng,
        a.Lat, a.Lng
    ) <= 5.0 -- Within 5 km
ORDER BY DistanceKm ASC;
```

Why This Algorithm is Effective

Criterion	Benefit
Proximity First	Patients get medicine faster; reduced delivery time
Distance-Based	Objective metric; no bias; repeatable results
Top 5 Results	Users have choice but not overwhelmed
Geographic Range	5km threshold balances coverage and practicality
Database Indexed	Lat/Lng can be spatially indexed for performance

Part 5: Example Scenario

Real-World Example: Patient Gets Prescription in Algiers

Step 1: Doctor Issues Prescription

Doctor: Dr. Hassan Mohamed

Clinic Location: 36.7538° N, 3.0588° E (Algiers, Downtown)

Prescription: Aspirin 500mg × 10 tablets

Step 2: System Finds Nearest Pharmacies

Fetches all pharmacies in Algiers database

Calculates distance from clinic (36.7538, 3.0588) to each

Step 3: Distance Calculations

└ Pharmacy A (Ben Aknoun): 36.7618, 3.0647 → 1.2 km

└ Pharmacy B (Kouba): 36.7450, 3.0720 → 2.8 km

└ Pharmacy C (Mustafa Pasha): 36.7380, 3.0500 → 2.1 km

└ Pharmacy D (Riad): 36.7290, 3.0390 → 3.5 km

└ Pharmacy E (Sidi M'Hamed): 36.7200, 3.0200 → 5.2 km (OUT OF RANGE)

Step 4: System Recommends

1. Pharmacy A - Ben Aknoun - 1.2 km ★ Closest
2. Pharmacy C - Mustafa Pasha - 2.1 km
3. Pharmacy B - Kouba - 2.8 km
4. Pharmacy D - Riad - 3.5 km

Step 5: Patient Selects

Patient chooses "Pharmacy A - Ben Aknoun"
 Prescription.PharmacyId = A
 Patient receives medicine in < 1 hour

Part 6: Data Model in Database

Tables

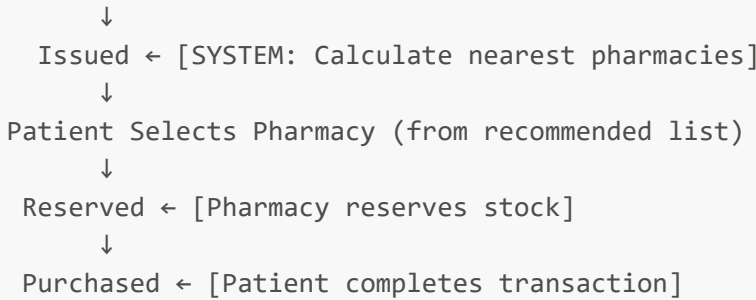
```
-- Address table
CREATE TABLE Address (
  AddressId INT PRIMARY KEY IDENTITY,
  UserId INT NOT NULL FOREIGN KEY REFERENCES [User](UserId),
  Wilaya NVARCHAR(100) NOT NULL,
  Province NVARCHAR(100),
  Commune NVARCHAR(100) NOT NULL,
  Rue NVARCHAR(255) NOT NULL,
  NumMaison INT,
  PlaceId NVARCHAR(255) UNIQUE, -- From Google Maps
  FormattedAddress NVARCHAR(500) NOT NULL,
  Lat DECIMAL(10, 8) NOT NULL, -- Latitude
  Lng DECIMAL(11, 8) NOT NULL, -- Longitude
  CreatedAt DATETIME DEFAULT GETDATE(),
  UpdatedAt DATETIME
);

-- Spatial Index for performance
CREATE SPATIAL INDEX IX_Address_Coordinates
ON Address (Lat, Lng);
```

Part 7: Integration with Prescription Flow

State Transitions

```
User Creates Prescription
  ↓
Draft
  ↓
Doctor Issues (with clinic location)
```



Key Data Flows

- 1. **Doctor Context** → Use Doctor's Address (clinic location)
- 2. **Patient Notification** → Use Patient's Address (show distance)
- 3. **Pharmacy Management** → Use Pharmacy's Address (inventory location)

Part 8: Configuration & Constraints

Configurable Parameters

Parameter	Default	Description
MaxRecommendationDistance	5 km	Maximum distance to recommend pharmacy
MaxRecommendationCount	5	Maximum pharmacies to return
MinPharmacyWilaya	-	(Optional) Restrict to same Wilaya only

Validation Rules

```
public class AddressValidator
{
    // Lat/Lng must be within Algeria's borders
    private const decimal AlgeriaMinLat = 18.9629m;
    private const decimal AlgeriaMaxLat = 37.0947m;
    private const decimal AlgeriaMinLng = -8.6676m;
    private const decimal AlgeriaMaxLng = 12.0305m;

    public bool IsValidAlgerianCoordinate(decimal lat, decimal lng)
    {
        return lat >= AlgeriaMinLat && lat <= AlgeriaMaxLat &&
            lng >= AlgeriaMinLng && lng <= AlgeriaMaxLng;
    }
}
```

Part 9: Performance Considerations

Optimization Strategies

1. **Spatial Indexing:** Create spatial index on (Lat, Lng) for $O(\log n)$ proximity queries
2. **Caching:** Cache frequently recommended pharmacies (refresh every 24 hours)
3. **Pagination:** Limit results to top 5-10 (avoid calculating all pharmacies)
4. **Async Calculation:** Calculate distances asynchronously if > 100 pharmacies

Query Performance

```
-- Without index: O(n) - scans all rows
SELECT * FROM Address WHERE Lat BETWEEN 36.7 AND 36.8;

-- With spatial index: O(log n) - fast lookup
CREATE SPATIAL INDEX IX_Coordinates ON Address(Lat, Lng);
```

Part 10: Future Enhancements

Potential Improvements

1. **Multiple Addresses:** Allow users to have home/work addresses
2. **Address History:** Track address changes over time
3. **Delivery Time Estimates:** Factor in traffic/delivery time (not just distance)
4. **Pharmacy Hours:** Recommend open pharmacies only
5. **User Preferences:** Learn user's preferred pharmacies
6. **Insurance Coverage:** Filter pharmacies covered by patient's insurance
7. **Real-time Stock:** Check medicine availability before recommending

Example Enhancement: Delivery Time

```
public decimal EstimateDeliveryTime(Address from, Address to, TimeOfDay timeOfDay)
{
    decimal distance = CalculateDistance(from, to);
    decimal speedKmPerHour = timeOfDay == TimeOfDay.Peak ? 15 : 25;
    decimal travelTime = distance / speedKmPerHour;
    decimal processingTime = 10; // minutes
    return (travelTime * 60) + processingTime;
}

// Result: "Pharmacy A - 1.2 km - ~12 min delivery"
```

Summary

The **Address entity** is designed as a hybrid system combining:

- **Administrative hierarchy** (Wilaya → Commune → Rue) for regional queries
- **Geospatial coordinates** (Lat/Lng) for distance calculations
- **Google Maps integration** (PlaceId/FormattedAddress) for accuracy

The **pharmacy recommendation algorithm** uses the Haversine formula to calculate distances and returns the 5 closest pharmacies within 5 km, enabling patients to receive prescriptions quickly and conveniently.

Appendix: Formula Reference

Haversine Formula (Full)

```
a = sin²(Δφ/2) + cos(φ1) × cos(φ2) × sin²(Δλ/2)
c = 2 × atan2(√a, √(1-a))
d = R × c
```

Where:

```
φ = latitude (in radians)
λ = longitude (in radians)
R = earth's radius (6,371 km)
d = distance
```

Coordinates Reference for Algeria

```
North (Mediterranean): ~37.0947°
South: ~18.9629°
West: ~-8.6676°
East: ~12.0305°
Center (Algiers): 36.7538° N, 3.0588° E
```